

SISTEMA DE AVALIAÇÃO COMPARATIVA MULTICRITÉRIO DE PRODUTOS (AHP-TOPSIS)

Desenvolvimento Kotlin/Android

FACOM32503 - Prof. Cláudio C Rodrigues

Sistema de Avaliação Comparativa Multicritério de Produtos (AHP-TOPSIS)

Objetivo Geral

O objetivo deste projeto acadêmico é desenvolver uma aplicação Android modular em **Kotlin** utilizando a arquitetura **MVVM** e interface em **Jetpack Compose**, capaz de realizar a avaliação comparativa de produtos por meio da integração dos métodos multicritério **AHP** (*Analytic Hierarchy Process*) e **TOPSIS** (*Technique for Order of Preference by Similarity to Ideal Solution*). A solução permite ponderar critérios com validação de consistência lógica (*RC <= 0,10*) e ranquear alternativas através de distâncias euclidianas às soluções ideais.

O projeto é uma das atividades avaliativas da disciplina de **Programação para Dispositivos Móveis** (**FACOM32503**) do Curso de Bacharelado em Sistemas de Informação da Universidade Federal de Uberlândia.

Contextualização do Tema

Em ambientes acadêmicos e mercadológicos, a seleção e comparação de produtos envolve a avaliação simultânea de múltiplos atributos frequentemente conflitantes — como preço, desempenho, durabilidade, qualidade e custos operacionais. Em diversos campos de atuação a tomada de decisão estruturada utiliza a **Análise de Decisão Multicritério (MCDA)** para transformar *trade-offs* subjetivos e dados brutos em conhecimento e escolhas racionais.

Na literatura científica, os métodos **AHP** (*Analytic Hierarchy Process*) e **TOPSIS** (*Technique for Order of Preference by Similarity to Ideal Solution*) se destacam como as técnicas mais consolidadas e empregadas para ranqueamento e seleção multicritério.

Definição do Problema de Pesquisa

Em processos tradicionais de comparação e escolha de produtos, o processo decisório frequentemente padece de três grandes limitações metodológicas:

1. **Subjetividade e dificuldade na definição de pesos:** Decisores encontram grande dificuldade técnica em atribuir diretamente porcentagens ou pesos de importância a múltiplos critérios sem incorrer em arbitrariedade ou contradições.
2. **Carga computacional e exaustão do AHP puro:** O método **AHP** original exige comparações pareadas (par a par) entre todas as alternativas para cada critério. Quando o número de produtos alternativos (m) cresce, o volume de matrizes de comparação torna-se inviável e exaustivo para o tomador de decisão.
3. **Ausência de técnica nativa de pesos no TOPSIS puro:** O algoritmo **TOPSIS** exige que os pesos dos critérios sejam informados previamente por uma fonte externa, pois não possui um mecanismo interno para derivar pesos ou validar a coerência do julgamento humano.

Pergunta de Pesquisa: *Como estruturar um modelo híbrido de apoio à decisão (AHP-TOPSIS) para a comparação de produtos multicritério que reduza a subjetividade na atribuição dos pesos dos critérios e mantenha a eficiência do ranqueamento diante de múltiplos produtos e atributos?*

Justificativa e Relevância

- **Relevância Científica:** A integração **AHP-TOPSIS** une as principais virtudes de ambos os métodos. O **AHP** resolve a elicitação dos pesos dos critérios com validação matemática de coerência lógica, enquanto o **TOPSIS** realiza a agregação compensatória por distância euclidiana, garantindo baixo esforço de comparação e alta capacidade de escala.
- **Relevância Prática:** A aplicação de modelos quantitativos multivariados e multicritério gera vantagem competitiva nas organizações, otimizando investimentos, contratações e escolhas de produtos com transparência e rastreabilidade.

Objetivos Principais:

- **Objetivo Geral**: Desenvolver uma aplicação em Kotlin, utilizando arquitetura MVVM e Jetpack Compose que implemente o modelo híbrido AHP-TOPSIS para apoio à tomada de decisão.
- **Modelagem Matemática**: Estruturar a matriz de comparação pareada de *Saaty* e validar a coerência lógica via Razão de Consistência (RC <= 0,10).
- **Algoritmo de Ranqueamento**: Implementar a normalização vetorial e o cálculo das distâncias euclidianas às soluções ideais positiva (A+) e negativa (A-).
- **Engenharia de Software**: Aplicar padrões modernos de desenvolvimento móvel/desktop, com separação clara de responsabilidades e interface reativa.

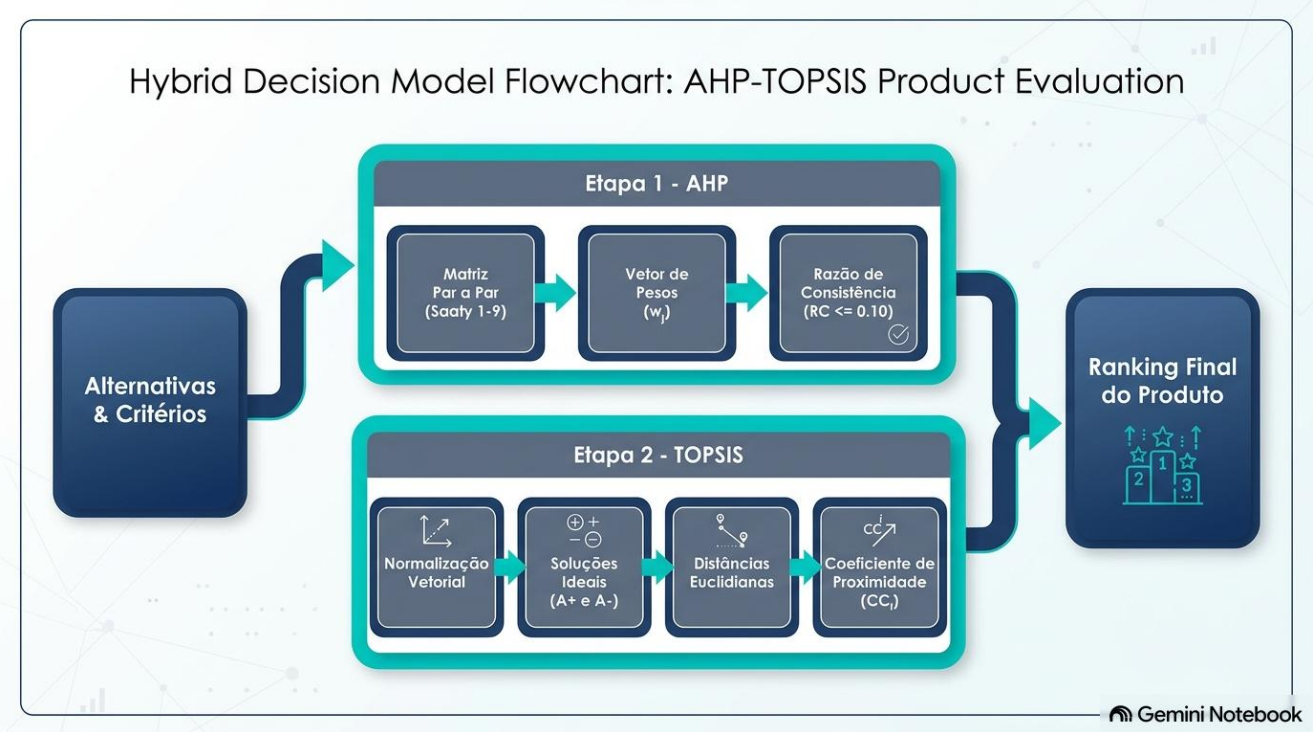
Objetivos do Desenvolvimento

- Aplicar conceitos de **desenvolvimento mobile Android** com **Jetpack Compose** e arquitetura moderna.
- Exercitar práticas de **gestão de projetos em equipe**, com definição de papéis e responsabilidades.
- Desenvolver competências em **modelagem de dados**, **arquitetura de navegação** e **design de interfaces**.
- Integrar o app com **bases de dados remotas**, simulando cenários reais de consumo de APIs.
- Consolidar o aprendizado em **testes funcionais e de usabilidade**, garantindo qualidade da solução.

Metodologia Proposta (Fluxo do Modelo Híbrido)

O procedimento metodológico divide-se em duas etapas integradas e sequenciais. O diagrama apresenta visualmente a sequência metodológica do projeto:

1. **Entrada de Dados:** Definição da categoria e dos produtos de análise (alternativas) e as características (critérios de benefício e custo).
2. **Etapa 1 – Ponderação (AHP):** Matriz de comparação pareada na **Escala de Saaty (1–9)**, cálculo do **vetor de pesos** dos critérios (w_j) e validação pela **Razão de Consistência** (RC <= 0,10).
3. **Etapa 2 – Ranqueamento (TOPSIS):** Normalização vetorial e ponderação da matriz, identificação das **Soluções Ideais Positiva** (A^+) **e Negativa** (A^-), cálculo das distâncias euclidianas (d^+ e d^-) e determinação do **Coeficiente de Proximidade Relativa** (CC_i).
4. **Saída:** Ordenamento e ranking final do produto vencedor.



Escopo do Projeto

- **Tela inicial do Sistema de Avaliação Comparativa**: Splash Screen apresentando o aplicativo com animação.
- **Lista das categorias de produtos**: cards com identificação visual da categoria de produto, nome da categoria.
- **Tela da Categoria**: cards com a coleção de produtos sob avaliação.
- **Tela de detalhes do produto**: foto, nome, modelo, fabricante, ...
- **Tela de definição dos pesos dos critérios de avaliação**
- **Telas de definição de avaliação de cada critério, para cada um dos produtos**
- **Tela com o resumo das configurações e botão de processar**
- **Tela de Resultados com ranqueamento TOPSIS e AHP, identificando (decoração gráfica) a melhor opção**
- **Construir a navegação estruturada (MVVM + NavHost)**: fluxo claro entre telas.
- **Integração com base remota**: dados acessados via API ou Firebase.
- **Gestão de projeto em equipe**: 8 alunos, sendo 1 líder responsável pela coordenação.

Estudo de Caso e Definição de Critérios

Definição do Produto: Seleção e Aquisição de Notebooks Corporativos para Equipes de Desenvolvimento de Software. O modelo avalia 4 alternativas de mercado frente a 5 critérios fundamentais com naturezas distintas (Benefício e Custo).

Produtos Avaliados (Alternativas):

1. **Notebook A:** Foco em altíssimo desempenho de processamento (Estação de Trabalho).
2. **Notebook B:** Foco em ultraportabilidade e bateria (Ultrabook Premium).
3. **Notebook C:** Opção de custo-benefício intermediário.
4. **Notebook D:** Foco em construção robusta e telas de alta resolução.

Características Avaliadas (Critérios do AHP-TOPSIS):

ID	Critério de Avaliação	Tipo	Unidade	Descrição
C_1	Desempenho (CPU/RAM)	Benefício	Pontos	Score quantitativo em benchmarks de processamento
C_2	Preço de Aquisição	Custo	R$	Valor total de compra (quanto menor, melhor)
C_3	Autonomia de Bateria	Benefício	Horas	Tempo estimado de uso contínuo longe da tomada
C_4	Peso / Portabilidade	Custo	Kg	Massa total do equipamento (quanto menor, melhor)
C_5	Qualidade de Construção	Benefício	Nota 1-10	Avaliação qualitativa de chassi, tela e teclado

Banco de Categorias de Produtos e Critérios

Para garantir a diversidade dos trabalhos desenvolvidos pelas equipes de trabalho, cada grupo de alunos deve selecionar, no mínimo, duas coleções de produtos distintas. Abaixo são apresentadas 4 categorias padronizadas com seus respectivos critérios de avaliação (qualitativos/quantitativos, benefício/custo):

Categoria / Domínio	Produtos (Alternativas)	Critérios de Avaliação	Tipo / Métrica
Cat. 1: Notebooks Corporativos (TI / Hardware)	• Notebook A (Workstation) • Notebook B (Ultrabook) • Notebook C (Intermediário) • Notebook D (Robustness)	C1: Desempenho CPU/RAM C2: Preço de Aquisição C3: Duração de Bateria C4: Peso / Portabilidade C5: Qualidade Construção	Benefício (Benchmark) Custo (R$) Benefício (Horas) Custo (Kg) Benefício (Nota 1-10)
Cat. 2: Smartphones Premium (Tecnologia / Consumo)	• Smartphone Flagship A • Smartphone B (Foco Câmera) • Smartphone C (Custo-Benefício) • Smartphone D (Bateria)	C1: Desempenho (Geekbench) C2: Preço de Venda C3: Qualidade de Câmeras C4: Autonomia de Bateria C5: Armazenamento Interno	Benefício (Pontos) Custo (R$) Benefício (Nota 1-10) Benefício (mAh / W) Benefício (GB)
Cat. 3: Veículos Elétricos (Automobilístico / ESG)	• Veículo Elétrico A • Veículo Elétrico B • Veículo Híbrido C • Veículo Elétrico D	C1: Autonomia Elétrica C2: Preço de Aquisição C3: Tempo Carga Rápida C4: Custo do Seguro Anual C5: Pacote Segurança ADAS	Benefício (Km) Custo (R$) Custo (Minutos 20-80%) Custo (R$) Benefício (Nota 1-10)
Cat. 4: Provedores de Nuvem (Infraestrutura B2B)	• Provedor A (AWS) • Provedor B (Azure) • Provedor C (GCP) • Provedor D (OCI)	C1: Custo Instância Padrão C2: Disponibilidade (SLA) C3: Latência Média Rede C4: Ecossistema / Suporte C5: Desempenho I/O Disco	Custo (USD/hora) Benefício (% Uptime) Custo (ms) Benefício (Nota 1-10) Benefício (IOPS)

Fundamentação Matemática dos Métodos Híbridos

A abordagem híbrida AHP-TOPSIS resolve duas limitações clássicas: a subjetividade direta na atribuição de pesos (solucionada pelo AHP) e a exaustão de comparações pareadas quando há muitas alternativas (solucionada pelo TOPSIS).

Etapa AHP (Determinação dos Pesos e Consistência)

1. Monta-se a matriz de comparação pareada A (n × n) usando a Escala Fundamental de Saaty (1 a 9).
2. Calcula-se o vetor de pesos w por meio da normalização das colunas e média das linhas.
3. Determina-se o Autovalor Máximo ($\lambda_{max} = \Sigma [(A \cdot w)_i / w_i] / n$).
4. Valida-se o Índice de Consistência ($IC = (\lambda_{max} - n)/(n - 1)$) e a Razão de Consistência ($RC = IC / IR$).
5. **Regra de Decisão:** Se $RC \leq 0{,}10$, os julgamentos são válidos. Caso contrário, reavalia-se a matriz.

Etapa TOPSIS (Ranqueamento e Distâncias)

1. Normalização Vetorial: $r_{ij} = x_{ij} / \sqrt{(\Sigma\, x_{kj}^2)}$.
2. Matriz Ponderada: $v_{ij} = r_{ij} \cdot w_j$ (utilizando os pesos w_j validados no AHP).
3. Identificação das Soluções Ideais Positiva (A+) e Negativa (A-):
   - Para critérios de Benefício: $A+ = \max(v_{ij})$, $A- = \min(v_{ij})$.
   - Para critérios de Custo: $A+ = \min(v_{ij})$, $A- = \max(v_{ij})$.
4. Cálculo das Distâncias Euclidianas: $d_i+ = \sqrt{(\Sigma\, (v_{ij} - v_j+)^2)}$ e $d_i- = \sqrt{(\Sigma\, (v_{ij} - v_j-)^2)}$.
5. Coeficiente de Proximidade Relativa: $CC_i = d_i- / (d_i+ + d_i-)$. Ordenamento decrescente por CC_i.

Requisitos Funcionais

- Entrada de matriz de comparação par a par para os 5 critérios.
- Cálculo automático do Autovalor Máximo (λ_{max}), Índice de Consistência (IC) e Razão de Consistência (RC).
- Alerta visual e bloqueio caso a matriz seja inconsistente (RC > 0.10).
- Cálculo da matriz normalizada e ponderada no TOPSIS.
- Apresentação das distâncias euclidianas (d_i^+ e d_i^-) e do ranking numérico final.

Requisitos Não Funcionais

- Interface intuitiva e responsiva.
- Código modular e bem documentado.
- Execução computacional instantânea dos cálculos matriciais.
- Controle de versão com **Git** em repositório compartilhado (GitHub/GitLab).
- Gestão de tarefas com ferramenta de apoio (ex: GitHub Projects).

Modelagem de Dados

Abaixo apresenta-se a estrutura das entidades do modelo para manipulação dos dados dos produtos e das matrizes dos métodos:

```
// Estrutura de dados das Alternativas (Produtos)
data class Product(
    val id: String,
    val name: String,
    val benchmarkScore: Double,  // C1: Desempenho (Benefício)
    val price: Double,           // C2: Preço (Custo)
    val batteryLifeHours: Double,// C3: Bateria (Benefício)
    val weightKg: Double,        // C4: Peso (Custo)
    val buildQualityScore: Double// C5: Qualidade (Benefício)
)

// Estrutura de resultado do Método Híbrido
data class DecisionResult(
    val product: Product,
    val distanceToIdeal: Double,     // d+
    val distanceToAntiIdeal: Double, // d-
    val closenessCoefficient: Double,// CC_i
    val rankPosition: Int
)
```

Arquitetura do Sistema e Organização da Equipe

O projeto segue a arquitetura de software recomendada para aplicações móveis/desktop reativas, organizando-se em 4 camadas principais: Camada de UI (Jetpack Compose), Camada de Negócio/Estado (ViewModel), Camada de Motores (Engine) e Camada de Dados (acesso a dados remotos).

Papel	Integrantes	Responsabilidades Principais
Líder do Projeto	1 Aluno	Coordenação geral, cronograma, integração de módulos e documentação.
Engenheiros de PO	2 Alunos	Modelagem matemática AHP/TOPSIS, testes matriciais e validação de consistência.
Devs Kotlin	2 Alunos	Implementação dos motores de cálculo, ViewModel, acesso ao repositório de dados e estruturação do código.
Designers / UX	1 Aluno	Prototipagem no Figma, telas Jetpack Compose e experiência visual do usuário.

Cronograma e Fases

Fase 1 – Requisitos + Modelagem (data de entrega: 03/11/2026)

- Definição dos requisitos funcionais e não funcionais.
- Modelagem da arquitetura do app (MVVM, camadas de dados e UI).
- Especificação dos layouts de tela e navegação.
- Entrega parcial: envio de link de vídeo descrevendo a etapa de desenvolvimento, inserção de documento com requisitos, diagramas de arquitetura e protótipos de tela no Teams.

Fase 2 – Desenvolvimento + Testes (data de entrega: 11/12/2026)

- Implementação das telas e navegação.
- Integração com base de dados remota.
- Testes funcionais e de usabilidade.
- Consolidação da versão final do app.
- Entrega final: apresentação do aplicativo funcionando, com demonstração prática.

Organização da Equipe

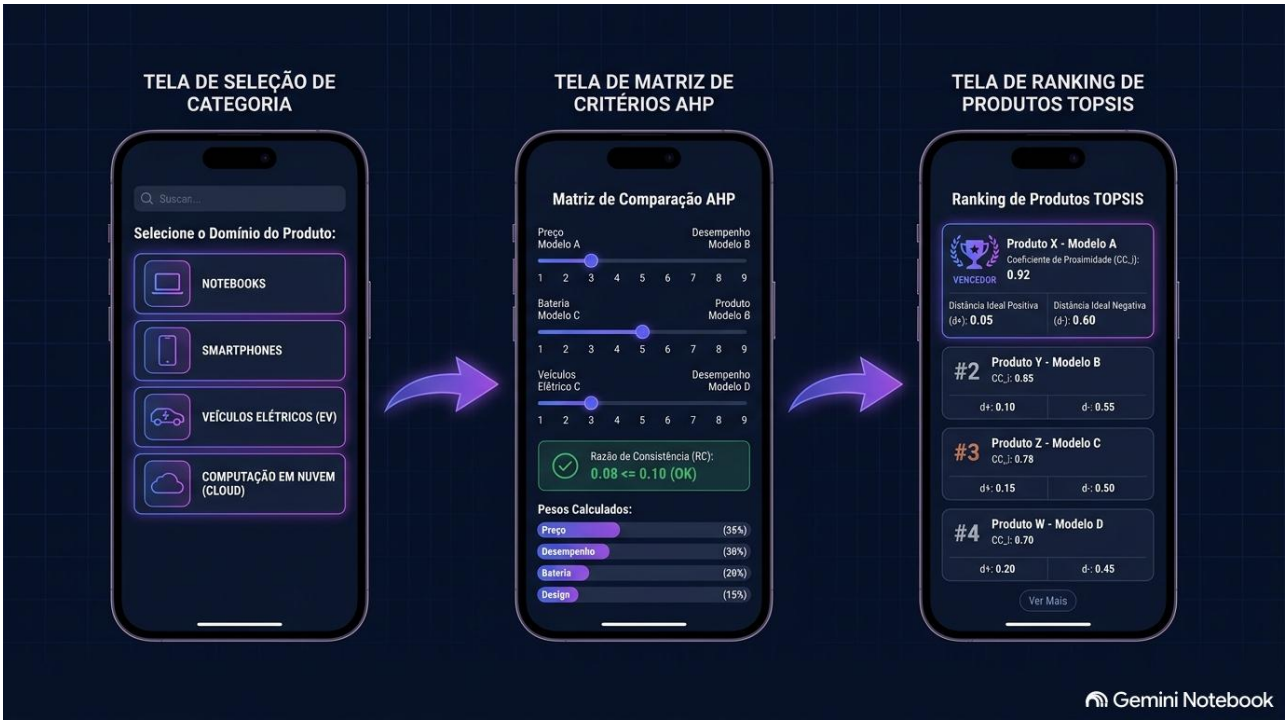
O projeto deverá ser desenvolvido por um grupo de até 6 componentes, sendo um dos componentes identificado como líder do grupo. Segue sugestão para a composição do grupo de trabalho:

Função	Quantidade	Responsabilidades Principais
Líder do Projeto	1	Coordenação geral, suporte no desenvolvimento, acompanhamento do cronograma, integração das entregas, comunicação com o professor e gestão de conflitos.
Desenvolvedores Android	2	Implementação das telas com Jetpack Compose, navegação entre telas, integração com ViewModel e testes de interface.
Engenheiros de Dados	2	Modelagem das entidades do projeto, criação e consumo da API remota, sincronização e cache local.
Designers/UX	1	Criação dos layouts, escolha de cores, ícones e tipografia, prototipagem no Figma e validação de usabilidade.

Critérios de Avaliação

- Clareza e completude dos requisitos.
- Qualidade da modelagem e arquitetura.
- Funcionalidade e usabilidade do app.
- Integração correta com base remota.
- Trabalho em equipe e gestão do projeto.
- Apresentação final clara e organizada.

✨ Esse documento fornece uma visão clara e estruturada da atividade, permitindo que os desenvolvedores sigam uma linha de gestão de projeto realista e organizada.



Roteiro para Apresentação em Vídeo

Fase 1 – Requisitos e Modelagem (03/11/2026)

Objetivo: demonstrar o planejamento e a estrutura do projeto.
Sugestão de sequência:

1. **Introdução (Líder do Projeto)**
   - Apresentar o tema "Sistema de Avaliação Comparativa Multicritério de Produtos".
   - Explicar o propósito da atividade e o contexto da competição.
2. **Requisitos (Engenheiros de Dados)**
   - Descrever os requisitos funcionais e não funcionais.
   - Mostrar como os dados serão obtidos de uma base remota (API ou Firebase).
3. **Modelagem (Desenvolvedores Android)**
   - Apresentar o diagrama de arquitetura MVVM.
   - Explicar a separação entre camadas (UI, ViewModel, Repositório).
   - Mostrar o diagrama de navegação entre telas.
4. **Design e Layouts (Designers/UX)**
   - Exibir protótipos das telas.
   - Justificar escolhas de cores, ícones e tipografia.
5. **Encerramento (Líder)**
   - Reforçar o cronograma e as entregas da próxima fase.

Fase 2 – Desenvolvimento e Testes (até 11/12/2026)

Objetivo: apresentar o aplicativo funcional e os resultados obtidos.
Sugestão de sequência:

1. **Introdução (Líder)**
   - Relembrar os objetivos e o progresso desde a Fase 1.
2. **Implementação (Desenvolvedores Android)**
   - Mostrar o app em execução.
   - Explicar o fluxo de dados entre as camadas (MVVM).
3. **Integração de Dados (Engenheiros de Dados)**
   - Demonstrar a comunicação com a base remota.
   - Exibir exemplos de requisições e respostas.
4. **Testes e Usabilidade (Designers/UX)**
   - Apresentar resultados de testes funcionais e feedback de usuários.
5. **Conclusão (Líder)**
   - Destacar aprendizados, desafios e melhorias futuras.

Cronograma Simplificado do Projeto – Álbum de Figurinhas

Abaixo é apresentado uma **sugestão** de **modelo de cronograma visual (estilo Gantt simplificado)** e orientações para o acompanhamento das atividades do grupo.

Fase	Período	Atividades Principais	Responsáveis	Entregas
1. Planejamento e Requisitos	26/09 → 12/10	- Definição dos requisitos funcionais e não funcionais - Levantamento das fontes de dados remotas - Distribuição de papéis e tarefas	Líder + Engenheiros de Dados	Documento de requisitos e plano de equipe
2. Modelagem e Prototipagem	09/10 → 26/10	- Modelagem da arquitetura MVVM - Criação dos diagramas de navegação e fluxo de dados - Prototipagem dos layouts no Figma	Designers/UX + Desenvolvedores Android	Documento técnico e protótipos de tela
3. Desenvolvimento e Integração	27/10 → 27/11	- Implementação das telas e navegação - Integração com base de dados remota - Testes unitários e de integração	Desenvolvedores Android + Engenheiros de Dados	Versão funcional do app
4. Testes e Consolidação	28/11 → 07/12	- Testes de usabilidade e correções - Ajustes finais de layout e desempenho - Preparação da apresentação final	Todos os membros	App final e relatório de testes
5. Apresentação Final	08/07	- Demonstração do app - Exposição dos resultados e aprendizados	Líder + Equipe	Apresentação oral e entrega final

Orientações para o Processo de Desenvolvimento

1. Planejamento semanal: o líder deve realizar reuniões curtas (15–20 min) para revisar progresso e ajustar tarefas.
2. Controle de versão: cada membro deve trabalhar em *branches* separados e realizar *pull requests* revisados pelo líder.
3. **Documentação contínua:** manter README atualizado com instruções de compilação e uso.
4. **Testes incrementais:** validar cada módulo antes da integração final.
5. **Apresentação final:** preparar slides com os diagramas (navegação, fluxo de dados, MVVM) e demonstração prática do app.

✨ Exemplo de estruturação de dados para o Sistema de Avaliação Comparativa Multicritério de Produtos, codificado em XML.

Estrutura do Dataset Gerado (dados-categorias-ahp-topsis.json)

```
{
  "categories": [
    {
      "id": "notebooks_corporativos",
      "name": "Notebooks Corporativos para TI",
      "criteria": [ ... ],
      "pairwiseMatrix": [ ... ],
      "products": [ ... ]
    },
    {
      "id": "smartphones_premium",
      "name": "Smartphones Premium",
      "criteria": [ ... ],
      "pairwiseMatrix": [ ... ],
      "products": [ ... ]
    },
    {
      "id": "veiculos_eletricos",
      "name": "Veículos Elétricos de Entrada",
      "criteria": [ ... ],
      "pairwiseMatrix": [ ... ],
      "products": [ ... ]
    },
    {
      "id": "provedores_nuvem",
      "name": "Serviços de Infraestrutura em Nuvem (Cloud B2B)",
      "criteria": [ ... ],
      "pairwiseMatrix": [ ... ],
      "products": [ ... ]
    }
  ]
}
```

Exemplo de Integração em Kotlin (Repository.kt)

```
// Você pode carregar e desserializar esses dados
// via Retrofit ou leitura local no repositório:
import com.google.gson.Gson
import com.google.gson.reflect.TypeToken

data class CategoryData(
    val id: String,
    val name: String,
    val description: String,
    val criteria: List<Criterion>,
    val pairwiseMatrix: Array<DoubleArray>,
    val products: List<Product>
)

class DecisionRepository {
    fun loadCategoryData(jsonString: String): List<CategoryData> {
        val type = object : TypeToken<Map<String, List<CategoryData>>>()
{}.type
        val      map:      Map<String,      List<CategoryData>>      =
Gson().fromJson(jsonString, type)
        return map["categories"] ?: emptyList()
    }
}
```

✨ Destaques do Design e Arquitetura do Projeto

- **Padrão MVVM Reativo:** A UI consome o StateFlow exposto pelo DecisionViewModel, reagindo automaticamente a mudanças de estado (Idle, Success, Error).
- **Validação em Tempo Real:** O retorno visual do cálculo da Razão de Consistência (RC <= 0.10) bloqueia o avanço para a tela de ranking caso os julgamentos apresentem contradição lógica.
- **Hierarquia Visual e Contraste:** O produto vencedor ganha destaque com cores diferenciadas (*Gold Container*), elevação superior e o ícone de estrela, direcionando a atenção do decisor para a escolha ótima.

O design editorial do Sistema de Avaliação Comparativa Multicritério de Produtos deve criar ritmo, coerência e uma progressão envolvente.

Aspectos estéticos e identidade visual

A estética do projeto, que inclui paleta de cores, tipografia e estilo das ilustrações nas páginas de categorias e produtos, deve refletir a personalidade da marca e o tema. Texturas, acabamentos e formatos diferenciados valorizam ainda mais o produto final, tornando a interação com o sistema agradável.

Com base nos requisitos e diretrizes técnicas do projeto, aqui está uma proposta de sequência de *layouts* para as telas do aplicativo, projetada para uma navegação intuitiva em smartphones:

Modelagem e Prototipagem com Figma

No contexto do seu projeto, o Figma é a ferramenta central para a etapa de **Modelagem e Prototipagem**, sendo uma responsabilidade atribuída especificamente aos integrantes do papel de **Designers/UX**.
Para criar protótipos eficazes que atendam aos requisitos da disciplina, siga este roteiro:

1. Aplicação de Princípios de Design e Estética - Use as ferramentas de estilo do Figma para garantir a **unidade visual** e o **engajamento**:

- **Hierarquia e Contraste:** Utilize o tamanho e o contraste para **direcionar o olhar** do usuário. O que for maior ou tiver mais contraste na tela deve ser o que chama a atenção primeiro. Segundo as fontes, deve-se considerar que o lado direito pode atrair o olhar prioritariamente.
- **Espaço em Branco:** Não lote a tela de elementos. Use o "vazio" como um condutor natural do olhar para criar harmonia.
- **Tipografia:** Escolha fontes **não serifadas** para títulos e textos curtos, pois são menos cansativas em interfaces digitais, reservando serifas para guiar o olhar em textos mais longos, se houver.
- **Identidade da Marca:** Personalize cada tela de equipe com suas **cores oficiais** para criar um vínculo emocional e refletir a personalidade da seleção.

2. Prototipagem da Navegação (Interatividade) - O diferencial do Figma é a aba "*Prototype*", onde você deve simular o fluxo real do aplicativo:

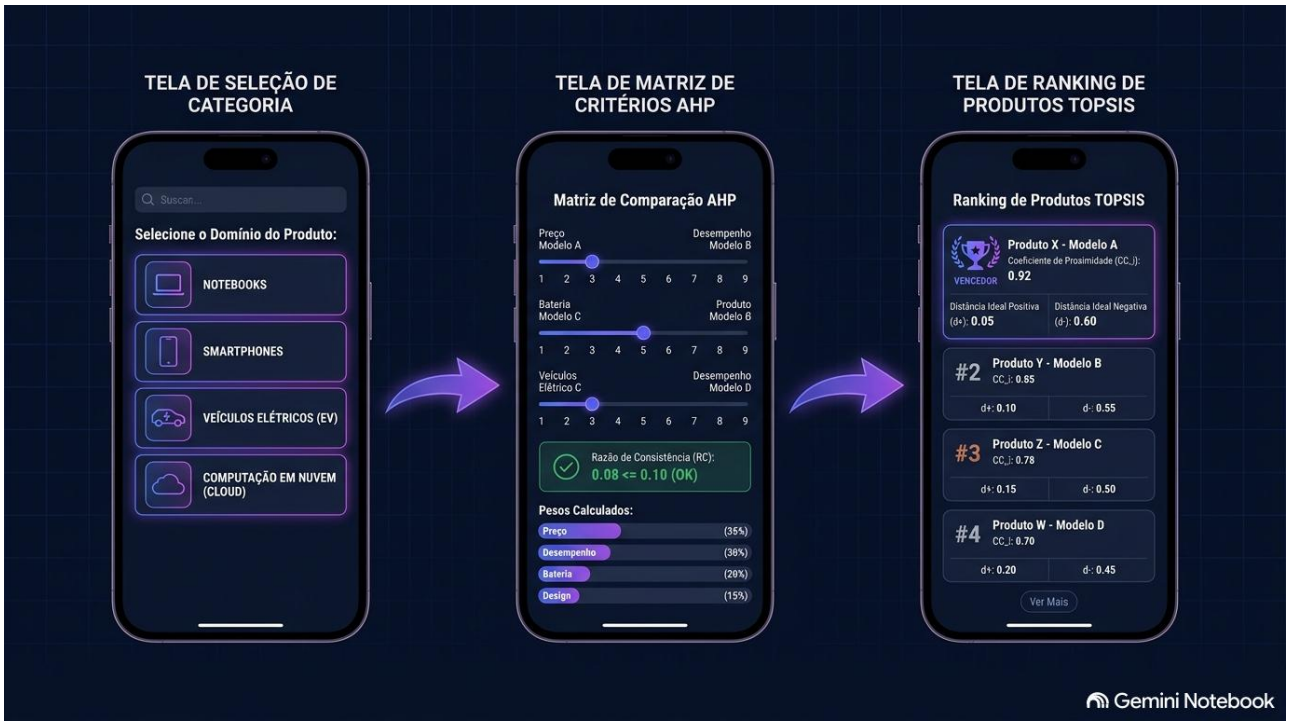
O protótipo Figma completo do Sistema de Avaliação Comparativa Multicritério (AHP-TOPSIS) define 8 telas mobile com navegação interativa:

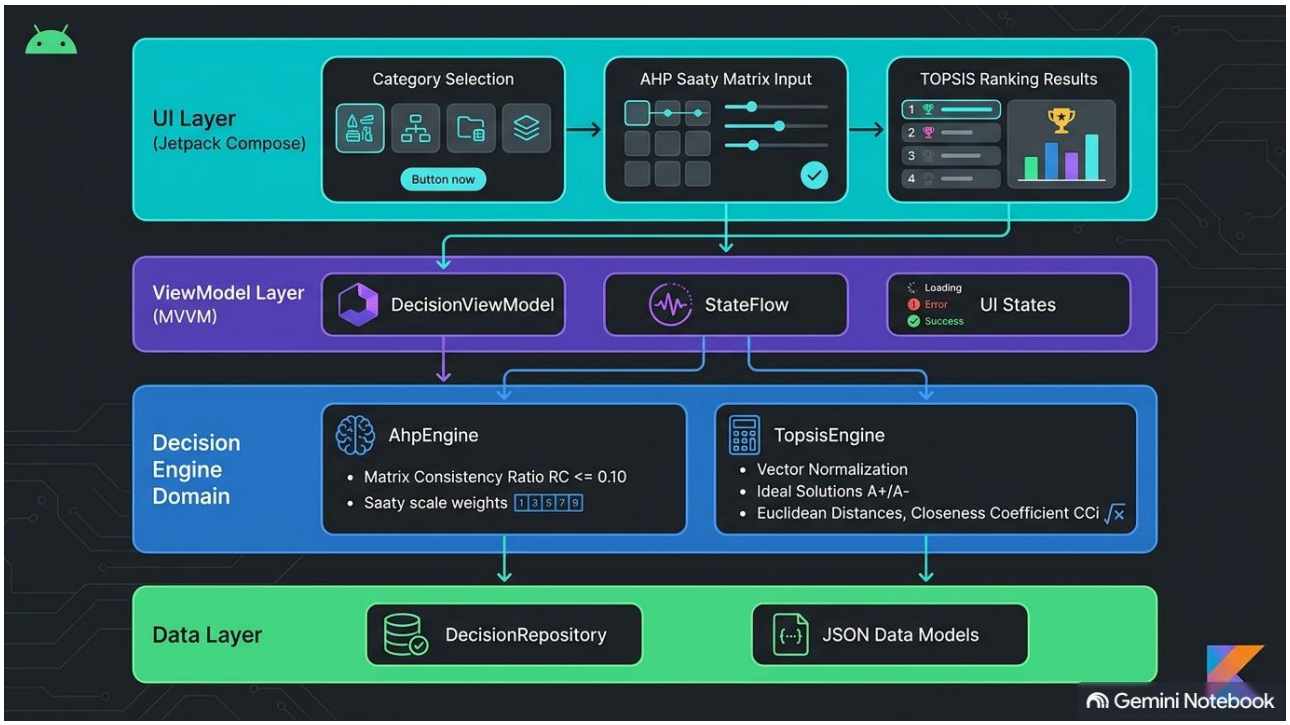
— Tela de abertura com logo e nome do app "CompareAI"
— Cards para Notebooks, Smartphones, Veículos Elétricos e Cloud
— Produtos da categoria com botão "Iniciar Avaliação"
— Comparação pareada com Escala de Saaty, Razão de Consistência e pesos calculados
— Input de valores para cada produto por critério
— Revisão dos pesos e matriz de decisão com botão "Processar TOPSIS"
— Classificação final com CCi, d^+ e d^- por produto
— Gráficos comparativos e matriz normalizada ponderada

O fluxo de protótipo está conectado: Splash (auto 2s) → Categorias → Produtos → AHP → Critérios → Resumo → Ranking → Detalhes → Nova Avaliação (volta ao início).

Link protótipo: https://www.figma.com/proto/i7GiRLROvuO6fA0CsUEhZO/Untitled?node-id=4-491&p=f&t=JSANCi57nLyi1qOX-0&scaling=scale-down&content-scaling=fixed&page-id=0%3A1

✨ Dica para a entrega: Esses protótipos são fundamentais para a **Fase 1 (Requisitos + Modelagem)**, onde o grupo deve apresentar os diagramas e as telas prontas antes de iniciar a codificação.





Estrutura da Arquitetura em Camadas (MVVM + Decision Engines)

O diagrama ilustra a organização modular do aplicativo em **4 camadas principais**, garantindo a separação de responsabilidades, testabilidade dos motores matemáticos e reatividade da interface:

1. **Camada de UI (Jetpack Compose & Material 3):**
   - **Função:** Renderiza as interfaces reativas do usuário, formulários interativos da Escala de Saaty (1 a 9) e *Cards* de resultado do ranking final.
2. **Camada de Apresentação e Estado (ViewModel - MVVM):**
   - **Função:** Mantém e expõe o estado reativo da aplicação por meio de StateFlow, recebendo ações da UI e orquestrando a execução dos motores de cálculo sem travar a *Main Thread*.
3. **Camada do Domínio / Motores Matemáticos (Engine AHP-TOPSIS):**
   - **AhpEngine:** Executa o cálculo do autovalor máximo, vetor de pesos e validação da **Razão de Consistência.**
   - **TopsisEngine:** Realiza a normalização vetorial, matriz ponderada, identificação das Soluções Ideais, distâncias euclidianas e o Coeficiente de Proximidade.
4. **Camada de Dados / Repositório (Data Layer):**
   - **Função:** Carrega e desserializa o arquivo de banco de dados JSON, provendo os dados de entrada para as 4 categorias de produtos.